

Issues — Open Workable Items

Revision: 12 **Last modified:** 2026-08-20T15:54:29Z **Ticket prefix:** BOB (operator-mandated, 2026-06-06) **Scope:** Open/active items only. Closed items migrate to [Fixed.md](#).

Tracking: this file + [Issues_Summary.md](#) are authoritative for open work. The SQLite single-source-of-truth + docs_chain engine (BOB-010) is complete.

BOB-008 — RuTracker automated login blocked by CAPTCHA

Status: Operator-blocked **Type:** Bug **Created:** 2026-06-06
Operator-Block-Details: WHAT — RuTracker login with stored creds returns no session cookie (CAPTCHA wall). WHY — automated user/pass login is CAPTCHA-gated; self-resolution exhausted (creds correct + wired, login attempted, auth=True). UNBLOCK — [A] operator completes the CAPTCHA flow at /api/v1/auth/rutracker/captcha + /login. [B] operator pastes a fresh bb_session cookie via /auth/rutracker/cookie-login. WHO — operator.

Evidence: live search per-tracker stat rutracker status=error auth=True. The diagnostic now reports error_type="upstream_captcha" with the FACT-based message "rutracker login.php returned no session cookie — this is the rutracker anti-abuse CAPTCHA wall (gates login.php when logins spike), not a credential failure. Set RUTRACKER_COOKIES from a logged-in browser session to bypass the login round-trip." (download-proxy/src/merge_service/search.py). The earlier quote here — error="login returned no session cookie — likely CAPTCHA" — was SUPERSEDED when BOB-117 landed and no longer exists in source; it is corrected rather than left as stale evidence (§11.4.6 no-guessing, §11.4.7 evidence must reflect the conditions it claims).

BOB-065 — Lava P2: Egress diagnosis and VPN-host SOCKS routing (containers pkg/egress)

Status: Queued **Type:** Task **Severity:** High

[Backfill from RD2-15/GA-05, audit doc 2026-08-08] Lava-porting finding P2 (Egress decision + VPN-host routing, Lava PLAYBOOK sections 0 and 4). Problem Boba-Base shares: on a datacenter host, trackers are network-blocked (DNS-fail/TLS-MITM, not Cloudflare so FlareSolvrr cannot fix). Affects Jackett indexer fetches + merge_service/download-proxy + plugin engines. Diagnosis (port the script): curl https://api.ipify.org (host IP) + curl -o /dev/null -w %{http_code} https:// direct vs via a VPN-host SOCKS proxy. Different egress IP + 200 via proxy confirms. Fix: route outbound through a VPN-connected host (the nezha pattern). SOCKS tunnel ssh -D 127.0.0.1:1080 -N (use -socks5-hostname for remote DNS); point Jackett + download-proxy + qBitTorrent-go at it (P3). For browser-cookie harvest, run the harvester ON the VPN host. Port: containers submodule pkg/egress (tunnel up/verify) + scripts/egress-via-vpn.sh glue; reuse Boba-Base existing ensure-macos-tunnel.sh style. TDD: assert the via-proxy egress IP != direct host IP AND a known-blocked tracker returns 200 via proxy. Source: docs/PORTING-FROM-LAVA.md. Per audit RD2-15 [P0]: Create tracked workable items (BOB-064..067) for the four Lava-porting findings, citing implementing commits as evidence, closed as Implemented.

BOB-066 — Lava P3: BOBA_UPSTREAM_PROXY in download-proxy + qBitTorrent-go + Jackett + compose env-forward

Status: In progress **Type:** Task **Severity:** High

[Backfill from RD2-15/GA-05, audit doc 2026-08-08] Lava-porting finding P3 (Configurable outbound proxy in the services, Lava PLAYBOOK section 3). download-proxy (Python): httpx/requests honor HTTP_PROXY/HTTPS_PROXY/ALL_PROXY/NO_PROXY env natively — add an explicit BOBA_UPSTREAM_PROXY config that sets these for tracker-bound clients, with loopback bypass (NO_PROXY=127.0.0.1,localhost,jackett). qBitTorrent-go: set http.Transport.Proxy (socks5 native, remote DNS) from a BOBA_UPSTREAM_PROXY env — mirror Lava internal/httpx/proxy.go. Jackett: has a built-in proxy setting (configure via its API/ServerConfig). Deploy gotcha (port): the env must be FORWARDED into the containers (docker-compose.yml env / the

boba-ctl deploy) — Lava bug was a missing allow-list entry. Verify on distroless via podman inspect, not exec printenv. TDD: a test with a local proxy asserts the service tracker request traverses it; falsifiability: disable the wiring so the test fails. Source: docs/PORTING-FROM-LAVA.md. Per audit RD2-15 [P0]: Create tracked workable items (BOB-064..067) for the four Lava-porting findings, citing implementing commits as evidence, closed as Implemented.

BOB-068 — RD2-00: unattributed, unreviewed Auto-commit mechanism pushing to main

Status: Queued **Type:** Bug **Severity:** High

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-00, Rev-6 downgrade to P1] Three commits at HEAD carry the bare message Auto-commit with no body, no ticket reference, no TDD trail: 54e313f (26 files), 9c8f684 (6 files), 743097a (2 files). Two landed while the audit was in progress. Source unidentified: no crontab, no systemd timer, no matching process, no in-tree script emitting the string, no hooksPath, no PostToolUse/Stop hook. Timezone +0500 on newer commits does not match host +0300 MSK. RD2-00 Update: git reflog proves 9c8f684/743097a arrived via pull: Fast-forward from a +0500 host, matching the established 2026-06-28 cross-host rsync-sync pattern (55b8671/cdb555f). Update 2 (Rev-6): root cause is a second live Claude session (Opus 5, same +0500 host) that independently landed constitution commit 177f2b0 (new anchor §11.4.238) while this session was mid-edit on the identical anchor number. Downgrades severity from P0 to P1. Every such commit bypasses §2 wrapper, §11.4.43 TDD-fix, §11.4.92 5-pass eval, §11.4.125/§11.4.142/§11.4.194/§11.4.209 mandatory Fable-xhigh review, §11.4.234 dedicated commit/push script. Composes with RD2-10..13.

BOB-069 — RD2-40: §11.4.238 QA-discovery-channel ledger — ongoing coverage-escape backfill

Status: Queued **Type:** Task **Severity:** High

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md Rev-6 addendum, priority downgraded P0->P1] §11.4.238 compliance MECHANISM STOOD UP AND EXERCISED 2026-08-08. docs/QA_DISCOVERY_LEDGER.md created with discovery-channel schema, seeded with 4 real entries from this session (RD2-22, RD2-41a, RD2-41b, BOB-008), each with a real

coverage-escape audit citing the specific missing/blind check; 3 of 4 already closed with a genuine new automated check carrying real RED->GREEN evidence. CLAUDE.md Critical Constraints now points at the ledger and states the discovery-channel-split mandate. Closed-loop example exercised end-to-end: scripts/pre_build_verification.sh invariant 17 extended to run workable-items diff (not just validate) — RED-captured via tests/unit/test_pre_build_workable_items_diff_check.sh §1.1 paired mutation. Honest boundary (§11.4.6): full retroactive audit of every one of the ~50 GA-NN/RD2-NN findings in this document (each needs its own escape-audit entry) was NOT attempted — ledger is honest that it starts now (100% out-of-band in its own tracked split table) rather than backfilling a false complete history. BOB-009/BOB-010 evidence_path gap remains open. Priority: downgraded from P0-blocking to P1-ongoing — MECHANISM exists and applies to new findings; the remaining retroactive audit + BOB-009/010 closure is incremental.

BOB-070 — RD2-41: pre-build mutation-marker scan carrier false-positive silently defeats entire pre-build gate

Status: Queued **Type:** Bug **Severity:** High

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md Rev-6 addendum, RD2-41 P1 — arguably P0] scripts/docs_chain.sh (FIXED this session) and scripts/pre_build_verification.sh invariant 17 (FIXED this session) both hardcoded a non-existent bin/workable-items path, silently no-op ing/skipping DB-export and DB-validate steps on every run — root-caused and fixed via the resolution chain proven in constitution/scripts/reporting/report_item.sh (env override -> committed constitution copy -> on-demand go build). Regression guards: tests/unit/test_docs_chain_binary_resolution.sh (RED->GREEN) + tests/unit/test_pre_build_workable_items_invariant.sh (RED->GREEN). A SEPARATE, still-open defect surfaced while fixing this: pre_build_verification.sh pre-code-review mutation-marker scan aborts the ENTIRE script (exit 1) before invariant 17 is ever reached, on carrier false-positives — legitimate constitution/**/*_mutation_test.sh + *_test.go files that intentionally contain the literal strings MUTATED/# MUTATION as part of their OWN §1.1 testing logic (36 hits, all under constitution/scripts/{gates,multitrack,workable-items})). Same §11.4.201 carrier-false-positive class as GA-24 + RD2-01. Practical impact: the ENTIRE pre-build gate has not completed a full run for as long as this false-positive has been live — no invariant past the mutation-marker check has been genuinely enforced. Fix needed: marker-

scanner to exclude self-referential test/gate files (or use structural check than bare substring), matching the fix scoped for GA-24/RD2-01/RD2-36. Priority: P1, arguably P0.

BOB-074 — RD2-07: DDoS-class testing fully absent from the mandated test-type matrix

Status: Queued **Type:** Task **Severity:** Medium

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-07, P2] tests/ has 18 subdirectories covering unit/integration/e2e/security/chaos/stress/performance/benchmark/UI (13 of the constitution ~14 mandated categories present). tests/load/locustfile.py covers throughput/scale only. No file anywhere mentions DDoS, flood, or attack-resilience testing — a distinct mandated category per §11.4.27, and not legitimately N/A given this is an internet-facing FastAPI/Gin service with public HTTP endpoints reachable over a LAN tunnel. Fix: author tests/ddos/ (or extend tests/load/) with rate-limit-enforcement, malformed-request-flood, and resource-exhaustion-under-attack coverage for the exposed download-proxy/merge-service endpoints. Priority: P2. Canonical implementation: RD2-32.

BOB-077 — RD2-10: Identify second host running the Auto-commit rsync/sync mechanism (OPERATOR-DECISION)

Status: Operator-blocked **Type:** Task **Operator-Block-Details:** WHAT: OPERATOR must identify which second host holds push credentials + confirms whether the rsync/sync mechanism is intentional or a stale job to retire. This session cannot inspect a host it has no access to (§11.4.6/§11.4.101). WHY: This session cannot inspect or reach the +0500 host that produced the Auto-commit fast-forwards (§11.4.6 no-guessing, §11.4.101 reversible-safe default). UNBLOCK: [A] operator names the second host + confirms intentional (proceed to RD2-11 wiring) · [B] operator confirms stale/misconfigured (retire the job) · [C] operator confirms the second live Claude session/device is the source (Rev-6 Update 2 root cause, proceed to per-session discipline enforcement) WHO: Operator **Severity:** High

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-10, P0 OPERATOR-DECISION] Root-caused as far as this host allows (see RD2-00 Update): reflog proves the two newest commits arrived via plain pull: Fast-forward from a +0500 host,

matching this project established 2026-06-28 cross-host rsync-sync pattern (cdb555f/55b8671). Needs operator input to go further — which second host runs it, and is it the intended mechanism (just needs a real message + review gate) or a stale job to retire. Not auto-executed (§11.4.6/§11.4.101 — this session cannot inspect or guess at a host it has no access to). Rev-6 update: root cause narrowed to a second live Claude session (Opus 5, same +0500 host) that independently landed constitution 177f2b0. Priority: P0 (operator-blocked).

BOB-078 — RD2-11: Once identified, wire Auto-commit mechanism through §11.4.234 dedicated commit/push script OR retire it

Status: Queued **Type:** Task **Severity:** High

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-11, P0] Once identified (RD2-10): either wire it through a real §11.4.234 dedicated commit/push script (with the hook-validation stages this project already has via .claude/settings.json PreToolUse guard, extended to a real pre-commit content check) or shut it down if it serves no purpose. Priority: P0. Blocked on RD2-10.

BOB-079 — RD2-12: Retroactive attributed history notes for GA-18/21/22/25/26/27 changes (never rewrite published history)

Status: Queued **Type:** Task **Severity:** Medium

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-12, P1] Retroactively give proper, attributed, reviewed commit messages / history notes to the technically-correct changes the Auto-commit mechanism already made (GA-18, GA-21, GA-22, GA-25, GA-26, GA-27) — this can be a documentation/changelog note rather than a history rewrite (§11.4.113 — never rewrite published history), citing the git-history evidence per §11.4.124 investigate-before-attribute pattern. Priority: P1.

BOB-080 — RD2-13: Retroactive Fable-xhigh code review of the substantive Auto-commit diffs

Status: Queued **Type:** Task **Severity:** High

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-13, P1] Retroactively run the mandatory independent Fable-xhigh code review (§11.4.125/§11.4.142/§11.4.209) against the substantive diffs the Auto-commit mechanism introduced (start.sh three new functions especially, since they have zero test coverage — see Root Cause 4 / RD2-24) — even though the changes already landed, a post-hoc review closes the governance gap and will surface RD2-24 (GA-27 missing tests) if not already caught. Priority: P1.

BOB-081 — RD2-14: Author CONTINUATION.md Session 15 entry (currently 53 days / 24+ commits behind HEAD)

Status: Queued **Type:** Task **Severity:** High

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-14, P0 — closes GA-04] Author a new CONTINUATION.md Session 15 entry summarizing every substantive commit since 646b295 by theme (Cyrillic/UTF-8 fix wave, tracker/plugin fixes, egress/proxy/Jackett feature wave, the two governance audits, RD2-00 discovery). Update **Last commit:/Last modified:/Revision:**. GA-04 evidence: still Revision: 19, Last modified: 2026-06-16T23:55:00Z, Session 14 — ~53 days / 24+ commits behind HEAD. Priority: P0.

BOB-082 — RD2-15: Create BOB-064..067 workable items for the four Lava-porting findings (closes GA-05)

Status: Queued **Type:** Task **Severity:** High

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-15, P0 — closes GA-05] Create tracked workable items (BOB-064..067, via the workable-items tool — never raw MD edits) for the four Lava-porting findings, citing implementing commits as evidence, closed as Implemented. GA-05 evidence:

grep -in lava|BOB-06[4-7] docs/Issues.md docs/Fixed.md returns zero hits. Priority: P0. NOTE: BOB-064..067 have been created 2026-08-10 as Task/Queued (the four Lava P1..P4 items); the closed-as-Implemented status flip (citing per-finding implementing commits) remains owed under this item.

BOB-083 — RD2-16: Regenerate browser_extension/features/codegraph Status.md + Summary/HTML/PDF siblings

Status: Queued **Type:** Task **Severity:** Medium

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-16, P1] Regenerate docs/browser_extension/Status.md, docs/features/Status.md, docs/codegraph/Status.md + their Status_Summary.md/HTML/PDF siblings (§11.4.45/§11.4.56). Priority: P1. Composes with RD2-08.

BOB-084 — RD2-17: Reconcile BOB-008 DB/MD body drift via the workable-items tool

Status: Queued **Type:** Task **Severity:** High

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-17, P1] Reconcile BOB-008 DB/MD body drift (RD2-04) via the workable-items tool. Priority: P1. Composes with RD2-04 (evidence source) and RD2-20 (preventive fix).

BOB-085 — RD2-18: Create top-level Boba proxy/merge-service v1.0.0 readiness ledger (closes GA-10)

Status: Queued **Type:** Task **Severity:** Medium

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-18, P2 — closes GA-10] Create the top-level Boba (proxy/merge-service) v1.0.0 readiness ledger (GA-10) — dedupe with the browser_extension existing one as the template. GA-10 evidence: only docs/RELEASE_READINESS_20260616.html/.md/.pdf (dated point-in-time snapshot) and the extension own ledger exist; no top-level proxy/merge-service ledger created. Priority: P2.

BOB-086 — RD2-19: Fix BOB-009/ BOB-010 evidence_path + backfill item_history for 56 silent closures

Status: Queued **Type:** Task **Severity:** Medium

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-19, P2] Fix BOB-009/BOB-010 evidence_path values + backfill item_history audit trails for the 56 silent closures where recoverable from git log (RD2-03). Rev-6 note: no workable-items subcommand exists to fix a historical closure evidence_path — see Root Cause 2 in the audit. Priority: P2.

BOB-087 — RD2-20: Wire docs_chain / commit-seam sync hook per §11.4.106(F) so DB writes cannot land without MD mirror

Status: Queued **Type:** Task **Severity:** High

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-20, P0] Wire (or fix) the docs_chain / commit-seam sync hook per §11.4.106(F) so a docs/workable_items.db write can never again land without its MD mirror in the same commit — this is the mechanical fix that prevents Root Cause 2 from recurring, not just a one-time catch-up. Priority: P0.

BOB-088 — RD2-21: Complete/verify README Tracked-Items + Status Documents table row-completeness (GA-07 remainder)

Status: Queued **Type:** Task **Severity:** Medium

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-21, P1] Complete/verify the README Tracked-Items + Status Documents table row-completeness (GA-07 remaining half). Not independently re-verified this round whether every mandated doc (CONTINUATION.md, Issues.md, Fixed.md, PORTING-FROM-LAVA.md, both new GA/RD2 audit docs, every Status.md/Status_Summary.md pair) actually has a row. Priority: P1.

BOB-089 — RD2-24: RED-first tests for start.sh reload_python/reload_plugins/recreate_stack (closes test-half of GA-27)

Status: Queued **Type:** Task **Severity:** High

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-24, P1 TDD — closes test-coverage half of GA-27] Author RED-first tests for start.sh reload_python(), reload_plugins(), recreate_stack() — real executing tests through the actual invocation path (per §11.4.224(A): never a bash -n parse-check alone), asserting exit status + the documented side effects (pycache-clear-then-restart, restart-only, down-then-up) against a live or realistically-mocked container runtime. Verification: tests fail against a stubbed/reverted version of the three functions, pass against current start.sh (§11.4.115). GA-27 evidence: grep -rln reload_python|reload_plugins|recreate_stack tests/ challenges/ returns zero hits — shipped with no test-first coverage, a direct §11.4.224 violation. Priority: P1.

BOB-090 — RD2-25: HelixQA Challenge entry exercising all three start.sh subcommands end-to-end against real compose stack

Status: Queued **Type:** Task **Severity:** Medium

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-25, P2] Add a HelixQA Challenge entry exercising all three start.sh subcommands (reload_python, reload_plugins, recreate_stack) end-to-end against the real compose stack. Priority: P2.

BOB-092 — RD2-27: Remove test_live_stack_evidence.py:265 nnmclub SKIP-on-404 fallback + verify live 200 (closes GA-13)

Status: Queued **Type:** Bug **Severity:** Medium

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-27, P1 — closes GA-13] tests/e2e/test_live_stack_evidence.py:265 — with the live stack up, confirm

curl :7187/api/v1/auth/nnmclub/status returns 200 (redeploy first if source≠artifact drift persists — see Root Cause 6), remove the skip fallback entirely, let the real assertion run. Priority: P1.

BOB-093 — RD2-28: Live compose bring-up + verify rutracker ReDoS regex bounds deployed to container (closes runtime half of GA-12)

Status: Queued **Type:** Task **Severity:** High

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-28, P1 — closes runtime-verification half of GA-12] Bring up the live compose stack (./start.sh), run ./install-plugin.sh, verify grep {0,512} config/qBittorrent/nova3/engines/rutracker.py on the container, and capture timing of a large rutracker result page (<2s per the original RW-06 acceptance criterion). GA-12 status: DONE (source) at plugins/rutracker.py:140 confirmed {0,512}/{0,256}-bounded; runtime unverified (no live container stack running at investigation time). Priority: P1.

BOB-094 — RD2-29: Author tests/stress/test_tracker_fetch_stress_chaos.py with §11.4.85 fault injection

Status: Queued **Type:** Task **Severity:** Medium

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-29, P2, parallelizable] Author tests/stress/test_tracker_fetch_stress_chaos.py (download-proxy tracker-fetch + cookie-auth path, overlapping the already-fixed SSRF surface) with real §11.4.85 fault injection (process-kill, network-fault, resource-exhaustion — not just concurrency). Verification: each new chaos test is negation-proven (fails without the fault-tolerance code, passes with it), captured evidence per §11.4.5/§11.4.69/§11.4.107. Priority: P2. Composes with GA-20.

BOB-095 — RD2-30: Author tests/stress/test_scheduler_hooks_sse_stress_chaos.py for Go-side triangle

Status: Queued **Type:** Task **Severity:** Medium

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-30, P2, parallelizable] Author tests/stress/test_scheduler_hooks_sse_stress_chaos.py covering the Go-side internal/service/sse_broker.go + internal/api/hooks.go + internal/api/scheduler_api.go triangle. Verification: each new chaos test is negation-proven (fails without the fault-tolerance code, passes with it), captured evidence per §11.4.5/§11.4.69/§11.4.107. Priority: P2. Composes with GA-20.

BOB-096 — RD2-31: Extend qBitTorrent-go jackett_db_test.go with real process-kill/resource-exhaustion fault injection

Status: Queued **Type:** Task **Severity:** Medium

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-31, P2] Extend qBitTorrent-go/tests/integration/jackett_db_test.go with real process-kill / resource-exhaustion fault injection (currently concurrency-only per GA-20). Priority: P2.

BOB-097 — RD2-32: Author DDoS-class coverage for exposed download-proxy/merge endpoints (canonical impl of RD2-07)

Status: Queued **Type:** Task **Severity:** Low

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-32, P3] Author DDoS-class coverage (RD2-07) for the exposed download-proxy/merge endpoints. Priority: P3.

BOB-098 — RD2-34: Parametrize 20 hardcoded /Volumes/T7 paths in helixqa banks with PROJECT_ROOT (closes GA-23)

Status: Queued **Type:** Task **Severity:** Medium

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-34, P1 — closes GA-23] submodules/helixqa/banks/.yaml — *parametrize the 20 hardcoded /Volumes/T7/Projects/Boba/... paths with \$PROJECT_ROOT (GA-23) — lands in the submodule per §11.4.28 decoupling, not boba own tree. GA-23 evidence: grep -rn /Volumes/T7 submodules/helixqa/banks/.yaml returns 20 hits.* Priority: P1.

BOB-099 — RD2-36: Fix guard-forbidden-commands.sh substring-match false-positive class + add const033-poweroff-signal-triage carrier to EXCLUDE_PATHS

Status: Queued **Type:** Bug **Severity:** Medium

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-36, P2] Fix guard-forbidden-commands.sh substring-match false-positive class (RD2-01) — word-boundary/token-aware matching, not another EXCLUDE_PATHS band-aid; also add the newly-discovered docs/incidents/2026-08-07-const033-poweroff-signal-triage.md carrier to EXCLUDE_PATHS as an immediate stopgap (GA-24 residual). Priority: P2. Composes with RD2-01, GA-24 residual, and RD2-41 (same class in mutation-marker scan).

BOB-100 — RD2-39: Bump submodules/jackett one commit (canonical impl of RD2-09)

Status: Queued **Type:** Task **Severity:** Low

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-39, P3] Bump submodules/jackett one commit (RD2-09). Priority: P3.

BOB-101 — GA-19/RW-09: Is -profile go parity still a release goal? (gates RW-10..13) — OPERATOR-DECISION

Status: Operator-blocked **Type:** Task **Operator-Block-Details:** WHAT: OPERATOR must decide whether Go -profile go parity remains a v1.0.0 release goal; gates downstream RW-10..13. WHY: §11.4.66 forbids autonomous decision on release scope; §11.4.122 forbids silent removal of an existing capability. UNBLOCK: [A] operator states YES — parity remains a release goal (schedule RW-10..13 work) · [B] operator states NO — mark parity Obsolete/superseded per §11.4.90 (reason=feature-removed with operator citation, §11.4.122) WHO: Operator **Severity:** High

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md GA-19/RW-09, OPERATOR-DECISION] Confirmed unchanged: zero time. Ticker anywhere in qBitTorrent-go, no enricher package, zero exec.Command fan-out. Is -profile go parity still a release goal? Gates RW-10..13. Surfaced only, not auto-executed, per §11.4.66. Priority: OPERATOR-DECISION.

BOB-102 — RW-05: LAN-exposure threat model — bind tunnel 127.0.0.1 or keep 0.0.0.0? — OPERATOR-DECISION

Status: Operator-blocked **Type:** Task **Operator-Block-Details:** WHAT: OPERATOR must decide LAN-exposure posture: bind tunnel to 127.0.0.1 (loopback-only) OR keep 0.0.0.0 (LAN-reachable, relying on post-RD2-22 complete auth coverage). WHY: This is a security-posture policy call the agent cannot make autonomously (§11.4.66 operator-decision, §11.4.101 high-blast-radius reversible-only-if-explicit). UNBLOCK: [A] operator states BIND 127.0.0.1 (loopback-only; agent will change compose/service binding + regression-test LAN unreachability) · [B] operator states KEEP 0.0.0.0 (rely on §RD2-22 completed auth coverage; agent will add a permanent §11.4.135 LAN-auth regression guard) WHO: Operator **Severity:** Medium

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RW-05 ungrouped, OPERATOR-DECISION] LAN-exposure threat model — bind the tunnel 127.0.0.1 or keep 0.0.0.0 with the now-complete auth coverage (post Root Cause 3)? Priority: OPERATOR-DECISION.

BOB-104 — §11.4.238 followup: CodeGraph 1.5.0 nested-.gitignore regression challenge

Status: In progress **Type:** Task **Severity:** Medium **Created-By:** Claude

Coverage-escape followup (docs/QA_DISCOVERY_LEDGER.md, BOB-075 agent-code-reading finding, commit e6162f7): CodeGraph 1.5.0 (up from documented 0.9.9) walked into nested-.gitignore-excluded frontend/node_modules and extension/node_modules (32,260 files / 514,456 nodes vs the 2026-06-06 baseline of 509 files / 8,906 nodes) instead of honoring frontend/.gitignore + extension/.gitignore per git check-ignore -v. Author a challenge (challenges/scripts/codegraph_gitignore_honor_challenge.sh or equivalent, e.g. wrapping 'codegraph doctor -sniff-gitignore-honor' if that subcommand exists, else a real re-index + file-count assertion) with §11.4.115 RED_MODE polarity: RED_MODE=1 reproduces the blowup against the live nested-.gitignore tree, RED_MODE=0 asserts the resync stays within the documented baseline order of magnitude. Full evidence: docs/codegraph/Status.md lines 91-118. UPSTREAM FILED 2026-08-18: real upstream repo is github.com/colbymchenry/codegraph (npm package @colbymchenry/codegraph, confirmed via package.json + gh repo view; NOT vasic-digital/codegraph, correcting this ledger's original assumption). Issue: <https://github.com/colbymchenry/codegraph/issues/1567> (full reproduction evidence: git check-ignore -v re-verified first-hand; two bounded synthetic repro attempts up to 63 nested .gitignore files / 960 files against the actual installed v1.5.0 binary did NOT reproduce the blowup in isolation - honest negative result, root cause not pinned to a specific line in either scanning implementation). Draft PR (diagnostic only, not a behavioral fix): <https://github.com/colbymchenry/codegraph/pull/1568> - adds a logDebug() call so a future report can confirm which of the two independent gitignore-respecting code paths (git-ls-files-based vs filesystem-walk fallback) actually ran. Both PRs/issue filed via SSH (Hard Stop #2) from a fork at github.com/milos85vasic/codegraph. Status: In-progress pending upstream maintainer triage/merge - boba-side closure of this item still needs the RED/GREEN challenge script (§11.4.115) authored per the original scope, independent of upstream's response.

BOB-105 — §11.4.238 followup: mechanical §11.4.227(B) anchor-block- integrity check

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** Claude

Coverage-escape followup (docs/QA_DISCOVERY_LEDGER.md, meta-defect found during this session's §11.4.140/141 collision resolution, boba commit 136a22c + constitution commits 5ed8c80..e5f2891): §11.4.227(B) anchor-block-integrity (exactly-once heading per anchor per governance file, byte-identical lockstep across mirrors, no anchor-number collision) is fully specified in constitution prose but its own named gate CM-ANCHOR-BLOCK-INTEGRITY is explicitly 'gate-code = separate work item, NOT claimed shipped' — the §11.4.140/§11.4.141 double-mandate collision this session's Phase 0 resolved was verified entirely BY HAND (md5 of moved anchor bodies, manual grep counts pasted into a commit message), not by a runnable script. Propose + author a challenge (in this project's challenges/scripts/, since this project cannot edit the constitution submodule's own gate-code from this ticket) that greps every governance file (CLAUDE.md/AGENTS.md/QWEN.md/GEMINI.md/Constitution.md, wherever this project's constitution submodule checkout exposes them) for '### §11.4.NNN' heading occurrences and FAILs on >1-per-file-per-anchor or on two DIFFERENT anchor bodies sharing one NNN, per §11.4.227(B). Do NOT touch constitution-owned files while implementing this — the check reads them read-only.

BOB-106 — §11.4.238 followup: §11.4.84 quiescence-check helper for the unattributed auto-commit path

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** Claude

Coverage-escape followup (docs/QA_DISCOVERY_LEDGER.md RD2-00/BOB-068 entry, docs/GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-00): 20 bare 'Auto-commit' commits exist in this repo's git history (confirmed via git log -oneline -all -grep, e.g. 54e313f/9c8f684/743097a/de9270b/1c36777/41179c2/7c529ca) with no ATM-NNN reference and no TDD trail, landing via ordinary git pull fast-forward from a second session/host with push access to the same remotes (mismatched commit timezone vs the investigating host, per RD2-00 Update). §11.4.84 working-tree-quiescence has no mechanical guard on this path — no gate flags

a commit reaching main with a bare/templated message and no ticket citation. Author a §11.4.84 quiescence-check helper (e.g. challenges/scripts/no_unattributed_autocommit_challenge.sh) that scans the commit range since the last known-good release tag and FAILs on any commit message matching a closed bare/templated pattern (e.g. ^Auto-commit\$, ^sync:) with no ATM-NNN or task/PR reference, wired into scripts/pre_build_verification.sh or the §11.4.234 commit-push-all.sh entrypoint. BOB-068 (RD2-00) remains the tracking item for identifying/stopping the source; this item is specifically the new automated CHECK.

BOB-107 — §11.4.238 followup: pre-dispatch existence check for subagent task-brief source inputs

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** Claude

Coverage-escape followup (docs/QA_DISCOVERY_LEDGER.md SCRATCH-LOSS-2026-08-18 entry, evidence: .superpowers/sdd/task-phase1a-report.md line 21): the Phase 1a subagent (a1cc331d) discovered at task start that 5 source files its brief named as required reads (curriculum_amendment_plan_v1.md, ai_curriculum_modules_27_35_extracted.md, and 3 curriculum_analysis_modules_*.md gap analyses) were absent from the session scratchpad — root cause per that subagent's own investigation: the prior producer subagent (ae59171f) hit its session rate limit (a §11.4.147(e) API-quota crash) before writing them. curriculum_amendment_plan_v1.md remains absent from the live scratchpad as of this session's re-check. No mechanical check verifies a task brief's declared input files exist and are non-empty before the downstream consumer subagent is dispatched. Author a pre-dispatch precondition helper (project-side orchestration tooling, e.g. a small script the conductor runs before Task/Agent dispatch when a brief names required input paths) that fails closed with an actionable 'missing input, respawn the producer' message rather than silently letting a downstream agent proceed on absent evidence and fabricate content unsupported by its named sources.

BOB-109 — BOB-074 followup: scaling-class test coverage absent from mandated test-type matrix

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** Claude

docs/testing/test_type_matrix.md's §11.4.27 test-type audit found zero scaling-class coverage anywhere in the tree (no scaling-tagged directory, test file, or HelixQA bank distinguishes growing-dataset/tracker-count/concurrent-user scale-out from stress-under-burst). Scope at least one scaling dimension, e.g. tracker-count scale-out in merge search against challenges/helixqa-banks/boba-services.yaml's tracker set, or the qbittorrent-proxy-go -profile go swap, with a real measured baseline.

BOB-110 — BOB-074 followup: UX-class test coverage (accessibility/usability) absent

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** Claude

docs/testing/test_type_matrix.md's §11.4.27 test-type audit found UI functional coverage (Vitest + Playwright) but nothing framed around usability/accessibility/UX outcomes specifically. Scope an axe-core or equivalent accessibility pass over the Angular frontend, covering WCAG checks, keyboard-nav coverage, and screen-reader labeling.

BOB-111 — BOB-074 followup: configure real rate limiting for boba's 3 public HTTP endpoints

Status: In progress **Type:** Task **Severity:** High **Created-By:** Claude

Source inspection across the whole stack (2026-08-18) verified no rate-limit mechanism exists for :7185 (qBittorrent WebUI proxy), :7187 (merge search service), or :7189 (boba-jackett) - no slowapi/limiter/throttle import in download-proxy/src/, no rate-limit middleware in qBitTorrent-go/internal/middleware/ (only cors.go + logging.go) or internal/jackettapi/ (only auth/cors middleware tests, no rate middleware), and no nginx/reverse-proxy service in docker-compose.yml. Candidate remediations documented in docs/testing/ddos_resilience.md: nginx-in-

container reverse proxy with `limit_req_zone` (most portable, adds a container); a FastAPI `slowapi` dependency for the merge-search service; a Gin rate-limit middleware for boba-jackett following the existing `internal/middleware/` pattern. qBittorrent's own WebUI bandwidth-shaping settings were NOT verified to cover request-rate (only bandwidth) - do not assume they close this gap without checking.

BOB-113 — BOB-074 followup: add wrk to dev tooling for DDoS/load challenges

Status: Queued **Type:** Task **Severity:** Low **Created-By:** Claude

`wrk` is not installed on the development host; ApacheBench (`ab`) is, and was used as the primary DDoS-resilience-challenge load tool per the task brief's 'wrk or ab' fallback path, with a `curl-parallel-loop` as the actual per-status-code assertion mechanism (more precise than `ab`'s aggregate summary) and `ab`'s own output pasted as supplementary real-tool evidence. Install `wrk` (e.g. via the project's `setup.sh` or a documented `apt/build-from-source` recipe) so future load/DDoS challenges have a modern HTTP benchmarking tool with latency-percentile histograms available out of the box, matching the brief's stated primary preference.

BOB-114 — BOB-074 followup: self-validation golden-bad fixture for the rate-limit detector

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** Claude

`challenges/scripts/ddos_resilience_challenge.sh`'s `-self-validate` mode currently only ships a golden-bad fixture for the crash-resistance detector (per §11.4.107(10)/§11.4.201). The rate-limiting detector has no matching golden-bad fixture proving it would actually FAIL a synthetic no-rate-limit-enforced artifact, so an unvalidated rate-limit detector could silently pass a broken/absent rate-limit deployment. Add a synthetic fixture (e.g. a stub server that never returns 429/503 under burst) and assert the detector correctly reports the absence, closing the self-validation gap for this detector class.

BOB-118 — README.md python-tests badge claims 585 passing; pytest -collect-only measures 5235 (9x stale/wrong)

Status: Queued **Type:** Bug **Severity:** High **Created-By:** AI

README.md line 28 ships a green/success-colored shields.io badge 'python tests-585 passing', and line 333 repeats '585+ tests'. A real, non-destructive verification this session (timeout 90 nice -n 19 ionice -c 3 .venv/bin/python -m pytest tests/ -collect-only -q) collected 5235 tests, not 585 — roughly 9x higher, so the badge is either wildly stale or was never machine-derived (violates §11.4.259 CM-BADGE-MACHINE-DERIVED-SOURCE + §11.4.6 no-guessing). The README's own §11.4.259 provenance footnote (lines 40-57) explicitly discloses that the tests/vitest/plugins/merge/scan/license badges were 'carried forward unverified this session (out of this task's scope)' and directs readers to docs/TESTING.md 'for the current authoritative per-suite counts' — but docs/TESTING.md contains NO occurrence of '585' or '182' anywhere, so the cited authoritative source cannot actually corroborate the badge. The frontend vitest badge (182 passing) is also suspect: a source-level grep of it()/test() calls in frontend/src/**/*.spec.ts counts ~371, roughly 2x the claimed 182 (lower-confidence proxy than the pytest collect-only count, but consistent with the same staleness pattern). Found during a §11.4.6 bluff audit (docs/qa/task-bluff-audit/). Fix direction: wire a real badge-computer (README already flags this as an owed §11.4.197 gate-code item) that regenerates the badge from a real collect/run count, and add the actual per-suite counts to docs/TESTING.md so the cited authoritative source is real.

BOB-120 — 3rd forced-logout incident 2026-08-18 23:45:49 — SIGKILL user@1000 + preventive-timer-inside-user-slice architectural gap

Status: Queued **Type:** Bug **Severity:** Critical **Created-By:** AI

user@1000.service SIGKILLed at 23:45:49 (2h55m after BOB-116's 20:50:59 kill, same session lineage), cascade-killing gnome-shell/Xwayland/ssh/dbus/gnome-keyring/all MCP servers/all boba containers' host processes (73 Killing-process lines captured). SIGKILL source UNCONFIRMED per §11.4.6 (kernel OOM clean, systemd-oomd no entries, no lid events, HandleLidSwitch=ignore unchanged) — same honest boundary as BOB-116. NEW architectural finding: boba-resource-pressure-

check.timer (BOB-116/task-77's preventive remediation) is a systemd -user unit hosted inside user@1000.service itself, so it dies in the SAME SIGKILL cascade it exists to detect precursors to; captured fire history shows last completed run finished 22:57:58, next due no earlier than 23:52:58, so the timer was not overdue at the 23:45:49 kill — it structurally cannot outlive the scope it monitors, meaning no in-scope self-monitor can ever catch a kill that takes it down with it. Contributing observations: qbittorrent-proxy health-probe ConnectionResetError cascade every ~30s through 23:38-23:45 (last line 23:45:44.737, 5s before kill, correlation only, not established causal); all four boba-stack containers (qbittorrent/jackett/boba-jackett/qbittorrent-proxy) recreated with CreatedAt=23:45:52, 3s after the kill, before any human session existed, confirming container supervision survives independently of user@1000.service. Fix direction: an out-of-user-scope watchdog (root-level systemd timer or a -user unit escaped from the session scope) is required to close this class of gap by construction. Evidence: docs/qa/BOB-120/{journalctl_23-42_to_23-46.log,timer_fire_history.log,qbittorrent_proxy_socketserver_traces.log}. Full writeup: docs/incidents/2026-08-18-3rd-forced-logout.md. Cross-refs: BOB-116 (docs/incidents/2026-08-18-perceived-forced-logout-2nd.md), §12.12 (RLIMIT_NPROC awareness anchor from the 2026-07-07 1st incident), task #77 (the timer this incident found the limitation of), task #79 (SIGKILL-source attribution, still open after 3 incidents). Left Queued — closing requires the actual out-of-scope-watchdog architectural fix (a documentation-only close would be a §11.4.238 coverage-escape bluff).

BOB-121 — External watchdog for the forced-logout architectural gap (task #85, incident #3)

Status: Queued **Type:** Task **Severity:** Important **Created-By:** Claude

Phase 1 design-only proposal: the BOB-116/task-77 resource-pressure preventive systemd -user timer runs inside user@1000.service, the exact pool it monitors, so it cannot fire when that pool dies (proven by incident #3, docs/qa/BOB-120/, 22:42+22:57 fires then blocked 23:45:49-23:49:00). Recommends Option B (user crontab reusing pre-existing crond.service in system.slice, no new root service) kept alongside the existing timer, with Option A (new root-owned systemd unit) as escalation path if Phase 1.5 live cron/cgroup verification is adverse. Proposal: docs/proposals/external-watchdog-for-forced-logout-architectural-gap.md. Operator decision required per §11.4.66 before any implementation - NOT implemented in this task.

BOB-129 — Potential production slowapi/starlette defect flagged by Task 105 subagent

Status: Queued **Type:** Bug **Severity:** Medium

Task #105 subagent (fixing 9 slowapi test failures) reported honestly that the same slowapi/starlette incompatibility likely hits the production /search and /search/stream endpoints under real HTTP traffic — evidence: the FastAPI TestClient (which goes through the full ASGI middleware stack like real requests do) reproduces the same isinstance() failure pattern the 9 test failures exhibited. Not yet reproduced against the running boba stack because the qbittorrent-proxy container currently exposes no host ports (running on gluetun network stack). Recommended investigation: (1) confirm defect by triggering /search kickoff through gluetun network stack, (2) if reproduced, determine whether the fix belongs in production code (adding response: Response params) or a version pin (slowapi vs starlette compat) or a middleware refactor. §11.4.238 discovery-channel escape prevention: manual QA must NOT be the discoverer. §11.4.108 Layer 3 verification: needed on a clean deployment before any release.

BOB-131 — qbittorrent-proxy podman common crash — pre-existing, surfaced during BOB-129 investigation

Status: Queued **Type:** Bug **Severity:** Medium

BOB-129 subagent found qbittorrent-proxy container DEAD mid-investigation (podman common crash). Recovery via ./start.sh -no-build worked. Pre-existing, unrelated to BOB-129 slowapi work but surfaced by it. Investigation needed: (a) how long was container dead before discovery? (b) what triggered the common crash? (c) is there a §11.4.144 always-follow / §11.4.128 always-record signal we should add to detect this class earlier? Post-recovery: container now unhealthy (see BOB-132). §11.4.238 discovery-channel escape: was originally found by a subagent investigating something else, not by dedicated container health monitoring.

BOB-135 — Test isolation: test_list_hooks_after_create fails in bulk suite (Permission denied /config)

Status: Queued **Type:** Bug **Severity:** Low

Task #109 subagent found: tests/unit/
test_merge_api_route_contracts.py::TestHooksEndpoint::test_list_hooks_after_create fails when run in bulk suite order with 'ERROR api.hooks:hooks.py:102 Failed to save hooks: [Errno 13] Permission denied: /config'. Passes in isolation (2.02s clean). Root cause: full-suite ordering pollution — some earlier test leaves state that makes hooks try to write to /config (which the test env doesn't own). Pre-existing, unrelated to BOB-126/BOB-129 chain. Fix strategy: identify the polluting test, add teardown or use a proper tempdir fixture for hooks storage in the offending test.

BOB-136 — Closure seam does not bind: 4 tracker rows found stale in one sweep, and workable-items diff is blind to body_md drift

Status: Queued **Type:** Task **Severity:** High

WHAT. A single 2026-08-20 sweep found FOUR workable-item rows whose tracked state contradicted reality: BOB-076 (closed by commit 99a486e on 2026-08-15, whose message literally says "closes BOB-076", still Queued), BOB-117 (fix landed AND covered by a permanent guard assertion, still Queued), BOB-091 (implemented under d1a8479 / a3b16bc / 2a0b543, still Queued), and BOB-008 whose Evidence field still quoted a diagnostic string that BOB-117 removed from source.

CORRECTION 2026-08-20 (11.4.6). An earlier revision of this item claimed the enabling defect was that "workable-items diff is blind to body_md drift". That was WRONG and is corrected here rather than quietly edited. The engine is NOT blind. The FLAGLESS INVOCATION is:

```
workable-items diff --db docs/workable_items.db  
-> "diff: DB and Markdown are in sync"      (opens ZERO .md  
files)
```

```
workable-items diff --db docs/workable_items.db --issues docs/  
Issues.md --fixed docs/Fixed.md  
-> "~ BOB-008 body differs (md=1346 bytes db=1333 bytes)" ->  
"diff: 1 difference(s)"
```

Both measured on the same seeded body-only drift. An independent agent proved the mechanism with strace: the flagless form never opens any Markdown file. So this is a 11.4.201(6) FALSE-NULL of the CALLER, not a gap in the comparison logic. Importantly, `pre_build_verification.sh` invariant 17 already passes both flags, so the STANDING GATE was never blind — the drift above was not caused by a blind gate.

THE REAL DEFECTS, restated honestly. (1) The flagless form is a false-null trap: it reports a confident, reassuring “in sync” while checking nothing. Any caller using it is a green-reporting blind check. It should either default to the conventional `Issues.md`/`Fixed.md` paths or REFUSE with a clear error, never return a clean verdict for a comparison it did not perform. (2) Work commonly lands under commit messages that do not carry the ATM id (GA-14/15/16 for BOB-091), so nothing mechanically links a commit back to its row, and closure depends on a human remembering. 11.4.227: an anchor done state is its SEAM landing, not its TEXT landing.

ACCEPTANCE. (a) The flagless diff invocation no longer returns a clean verdict without reading Markdown, with a paired 1.1 mutation proving it. (b) A repo-wide grep confirms no caller uses the flagless form. (c) A sweep that flags any non-terminal item whose id appears in a merged commit message, so done-but-open rows are found mechanically rather than by a human noticing. (d) Honest boundary: this does not claim to prevent all drift.

RELATED. `scripts/hooks/docs-sync-commit-seam.sh` (added under BOB-087) now enforces doc/DB sync at the commit seam and passes the flags explicitly, plus an independent `body_md` re-parse oracle.

EVIDENCE. `docs/qa/BOB-117/closure-evidence.md`, `docs/qa/BOB-076/closure-evidence.md`, `docs/qa/BOB-091/closure-evidence.md`.

BOB-137 — Merge service on 7187 wedges while the same process still serves 7186 (GIL starvation by one spinning thread)

Status: Queued **Type:** Bug **Severity:** High **Created-By:** Claude

Reported-Via: §11.4.202 reporting directive bug on 2026-08-20T14:46:52Z **Reported-By:** Claude

What (the report, verbatim): The merge service on port 7187 wedges after hours of uptime: the port stays bound and the container keeps reporting “healthy”, but every HTTP request hangs until the client times out. Measured 2026-08-20 16:41 UTC+2 on a container up 3h53m:

curl http://localhost:7186/ -> HTTP 200 in 0.096s (proxy, same process) curl http://localhost:7187/ -> HTTP 000 after 6.0s (merge service, WEDGED)

Both ports are served by the SAME process (pid 2330069, fd 3 = 7186, fd 7 = 7187), so this is not a crashed worker - one loop inside a live process has stopped servicing requests while another in the same process is fine.

Thread state at the time of the wedge (/proc/2330069/task, 16 threads): - tid 2330529: state R, wchan 0 <- ONE thread spinning on CPU - tid 2330528: state S, wchan do_sys_poll <- the healthy 7186 poll loop - the other 14: state S, wchan futex_wait_queue

That is the GIL-starvation signature: a thread busy-looping in Python holds the GIL and every other thread queues on the GIL futex. 7186 survives because do_sys_poll releases the GIL; the 7187 async loop needs sustained GIL time to service a request and starves. Process CPU was 20.6% while idle.

Corroborating evidence: seven sockets held by the process sit in CLOSE-WAIT with unread bytes still in the receive queue (Recv-Q 162, 162, 162, 79, 1, 1, 1) - clients sent a request and hung up, and the app never read it or closed the fd. The listener had also accumulated an unaccepted backlog (Recv-Q 6 on the 7187 LISTEN socket) at first observation. The container log’s last line is 2h before the observation, so the service processed nothing in that window.

NOT fd exhaustion: only 48 of 16384 fds were open.

ROOT CAUSE NOT ESTABLISHED. All four while True loops in the service (routes.py:166, search.py:1169, streaming.py:161, streaming.py:359) are correctly bounded and awaited, so the spin is not a naive unslept loop. py-spy could not attach to name the spinning frame - the host has kernel.yama.ptrace_scope=1, which denies non-child attach. Per the §11.4.102 Iron Law no fix is proposed until the spinning frame is identified.

Next diagnostic step: obtain a Python stack. Either run py-spy INSIDE the container (musl wheel), or set ptrace_scope=0 for the duration of one dump, or add a SIGQUIT/fault_handler.register() dump hook to main.py so the running service can be asked for its own stacks without ptrace.

DISCOVERY CHANNEL (§11.4.238): found by an agent probing the host by hand while investigating an unrelated test-suite result – NOT by automated QA. This is a coverage escape in its own right; see the sibling item on the health check that was structurally incapable of observing it.

Affected scope / file-scope manifest: download-proxy/src/main.py, download-proxy/src/merge_service/, download-proxy/src/api/streaming.py, docker-compose.yml (qbittorrent-proxy)

Reproduction / context: Leave the qbittorrent-proxy container running for several hours with search traffic (a full tests/security run is sufficient). Then: `curl -max-time 6 http://localhost:7186/` returns 200; `curl -max-time 6 http://localhost:7187/` returns 000. Confirm with: `ss -ltnp | grep 7187` (unaccepted backlog) and `awk '{print $3}' /proc//task/*/stat` (one R thread, rest `futex_wait_queue`).

Acceptance criteria: The spinning frame is identified from a real Python stack dump (not inferred), the busy-loop is fixed at its root, and a regression guard proves 7187 still answers after a sustained-traffic soak. Evidence: before/after curl timings on both ports plus a thread-state census showing no permanently-R thread.

BOB-139 — SSE _client_gone() swallows every exception into ‘client still connected’, so a raising disconnect probe streams forever (fail-open)

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** Claude

Reported-Via: §11.4.202 reporting directive bug on 2026-08-20T14:47:35Z **Reported-By:** Claude

What (the report, verbatim): Both SSE generators in download-proxy/src/api/streaming.py (lines ~153 and ~351) guard their while True loop with this helper:

```
async def _client_gone() -> bool:
    if request is None:
        return False
    try:
        return await request.is_disconnected()
    except Exception:
        return False
```

The bare except Exception: return False means: if the disconnect check itself ever raises, the generator concludes the client is STILL CONNECTED and keeps streaming – forever. That is fail-OPEN on the only condition that terminates the loop, in a code path that holds a socket and a task.

This is the §11.4.252 fail-closed rule inverted: the safe default for “I cannot determine whether the client is gone” is to treat it as gone and stop streaming (the client can always reconnect – SSE is designed for that), not to keep an unbounded stream alive on an unresolvable signal.

It is also a §11.4.201(6) false-null: a raising probe and a genuinely-connected client return the identical False, so the loop cannot distinguish “client is here” from “I am blind”.

Corroborating observation (2026-08-20): seven sockets held by the merge-service process were sitting in CLOSE-WAIT with unread request bytes – clients had hung up and the server had not noticed. That is consistent with (though not yet proven to be caused by) this fail-open, and it is the reason this is filed rather than left as a style note.

HONEST BOUNDARY: not proven to be the cause of the 7187 wedge – both loops DO call await asyncio.sleep(poll_interval), so they yield and would not busy-spin. This is filed as a real defect on its own merits (an unbounded stream on an unresolvable signal, plus a leaked socket and task), not as the wedge’s root cause.

Affected scope / file-scope manifest: download-proxy/src/api/streaming.py (_client_gone at ~line 153 and ~line 351)

Reproduction / context: Monkeypatch Request.is_disconnected to raise, open an SSE stream, disconnect the client, and observe the generator never terminates (the loop keeps yielding and the task is never reclaimed).

Acceptance criteria: A raising disconnect probe terminates the stream (fail-closed per §11.4.252) rather than continuing it, AND a normally-connected client still streams uninterrupted (§11.4.201 both directions). Guard: a unit test for each SSE generator covering raise -> terminate and connected -> continue.

BOB-141 — CLAUDE.md claims the Go profile serves 7186/7187/7188 but its container binds only 7187 — doc contradicts the Dockerfile

Status: Queued **Type:** Task **Severity:** Low **Created-By:** Claude

Reported-Via: §11.4.202 reporting directive task on
2026-08-20T14:56:38Z **Reported-By:** Claude

What (the report, verbatim): CLAUDE.md's Architecture section states:

"qbittorrent-proxy-go (Go/Gin, opt-in via -profile go) - replaces the Python proxy on 7186, 7187, 7188"

The container cannot deliver that. Read from source rather than prose:

```
qBitTorrent-go/Dockerfile:16 CMD ["/app/qbittorrent-proxy"]  
(ONE binary) qBitTorrent-go/Dockerfile:15 EXPOSE 7187 7188  
(declares 2) cmd/qbittorrent-proxy/main.go r.Run(fmt.Sprintf(":%  
%d", cfg.ServerPort)) (binds 1) internal/config/config.go:58  
ServerPort = MERGE_SERVICE_PORT (default 7187)
```

So the qbittorrent-proxy-go container binds 7187 ONLY. webui-bridge is a SEPARATE binary (cmd/webui-bridge, /bridge/health, cfg.BridgePort) that this container never starts, and nothing in it binds 7186 either - although the compose service does set PROXY_PORT=7186 and BRIDGE_PORT=7188 in its environment, which reinforces the wrong impression. EXPOSE likewise declares a port nothing binds.

WHY THIS MATTERS BEYOND TIDINESS: this prose was used as the source of truth when first authoring config/served_ports.yaml, producing a manifest entry of [7186, 7187, 7188] for that service. The healthcheck gate built on it then FAILED a service whose healthcheck was already correct - a §11.4.201(1) false-positive refusal caused directly by trusting the doc over the Dockerfile. The manifest was corrected against source; the doc was not, and will mislead the next reader the same way.

Either the doc is wrong, or the Go container is under-provisioned relative to intent (it should also run webui-bridge and a 7186 listener). Determining WHICH is part of this task - do not simply reword the doc to match the current binary if the intended design was a three-port container.

Related: the Go service's compose block sets PROXY_PORT and BRIDGE_PORT that no process in the container consumes, and EXPOSE lists 7188 unbound. Whichever way the above resolves, those should agree with reality afterwards.

Affected scope / file-scope manifest: CLAUDE.md (Architecture + Port Map), docker-compose.yml (qbittorrent-proxy-go env/EXPOSE), qBitTorrent-go/Dockerfile

Reproduction / context: Read qBitTorrent-go/Dockerfile:16 (single CMD) against CLAUDE.md's 'replaces the Python proxy on 7186, 7187, 7188'; confirm cfg.ServerPort resolves to MERGE_SERVICE_PORT=7187 in internal/config/config.go:58.

Acceptance criteria: Doc and container agree, with the direction of the fix decided deliberately (correct the doc, or provision the container to match the documented intent). PROXY_PORT/ BRIDGE_PORT env and EXPOSE lines agree with what the container actually binds.

BOB-143 — Orphaned .worktrees/ dirs (46M, unresolvable gitdir) pollute gate scan scope and manufacture false BOB-126-class findings

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** Claude

Reported-Via: §11.4.202 reporting directive bug on 2026-08-20T15:08:18Z **Reported-By:** Claude

What (the report, verbatim): .worktrees/ci-split-workflows/ (13M) and .worktrees/completion-initiative-phase-0/ (33M) are ORPHANED: git worktree list reports only the main checkout, so neither is a registered worktree. Each contains a .git POINTER FILE whose target gitdir no longer exists, so git cannot resolve HEAD, branch, or status inside them - every query returns empty.

They are gitignored (.gitignore:122), so nothing tracks them and nothing will ever notice them drifting.

WHY THIS IS NOT COSMETIC: they pollute the scan scope of whole-tree gates and manufacture false findings. Measured 2026-08-20 by the §11.4.32 sweep:

- cm_test_mock_pid_explicit_int reported 2 violations at tests/unit/merge_service/test_deadline_tunable.py:44 - BOTH inside these orphaned trees. The MAIN tree's copy of that file is already hardened (it sets mock.pid = 12345 and patches os.killpg/os.getpgid) and passes the gate cleanly. The finding read as a live §11.4.263 / BOB-126-class defect and was not one.
- 6 of the 57 "missing anchor carrier" files flagged by the propagation gates were likewise .worktrees/**.

That is the §11.4.201(1) false-positive shape sourced from scan scope, and it costs real investigation time: a reader triaging “2 live BOB-126 violations” reasonably treats it as a host-safety emergency.

CLARIFICATION (established during triage, so the record is not alarming): these trees are NOT a kill(-1) vector. Their download-proxy/src/merge_service/ search.py contains ZERO os.killpg calls – they predate that cleanup code entirely – so there is no unguarded signal call to reach. Additionally pyproject.toml sets testpaths = ["tests"], so a plain pytest run does not collect from .worktrees/. No host-safety risk was found. The defect is scan-scope noise plus 46M of unreferenced disk.

WHY REMOVAL IS NOT DONE AUTONOMOUSLY (§11.4.122 / §11.4.124 / §11.4.101): because git cannot resolve their HEAD, it is NOT possible to prove their contents are merged into main. Deleting unprovable-provenance work is exactly the irreversible, operator-owned decision §11.4.122 reserves. Two options for the operator: (a) confirm removal (they are stale dev scratch dirs) – reversible only from backup, so a §9.2 pre-op backup should precede it; or (b) keep them and add .worktrees/ to the gate scan-scope exclusion list as a §11.4.224(E)-fenced, checked-in, justified entry.

Either way the exclusion list is the cheaper immediate mitigation and does not destroy anything.

Affected scope / file-scope manifest: .worktrees/ci-split-workflows/, .worktrees/completion-initiative-phase-0/, gate scan-scope config

Reproduction / context: git worktree list shows only the main checkout; git -C .worktrees/

log -1 returns empty (gitdir target missing). Run the §11.4.32 sweep and observe cm_test_mock_pid_explicit_int report 2 violations, both under .worktrees/, while the main-tree file passes the same gate.

Acceptance criteria: Whole-tree gates no longer report findings sourced from orphaned worktrees: either the dirs are removed after operator confirmation with a §9.2 pre-op backup, or .worktrees/ is added to a checked-in §11.4.224(E)-fenced exclusion list with justification. Verify by re-running the sweep and confirming zero .worktrees-sourced findings.

BOB-144 — /theme/stream calls the disconnect probe unguarded — fail-closed but via an uncaught traceback, inconsistent with the two SSE generators

Status: Queued **Type:** Bug **Severity:** Low **Created-By:** Claude

Reported-Via: §11.4.202 reporting directive bug on 2026-08-20T15:53:57Z **Reported-By:** Claude

What (the report, verbatim): /theme/stream in download-proxy/src/api/routes.py:167 calls the disconnect probe with NO guard at all:

```
while True:
    if await request.is_disconnected():
        break
```

If that probe raises, the generator dies with an UNCAUGHT exception.

IMPORTANT — this is NOT the BOB-139 fail-open. The effect here is fail-CLOSED: the stream stops, and the enclosing finally: store.unsubscribe(queue) still runs, so the subscriber queue is released and nothing leaks. The outcome is CORRECT; the manner is not.

What is wrong with it: - it terminates via an uncaught traceback rather than a clean SSE close event, so the client sees a truncated stream instead of a reason; - the failure is not logged as a probe failure, so a systematically raising probe would show up as recurring tracebacks with no diagnosis; - it is inconsistent with the two SSE generators in streaming.py, which after BOB-139 emit event: close with reason disconnect_probe_failed and log a warning. Three call sites of the same probe now behave two different ways.

The BOB-139 fix deliberately did not touch this file (ownership boundary, §11.4.119), and flagged it honestly rather than fixing it out of scope.

Acceptance: /theme/stream uses the same probe-failure discipline as the streaming.py generators — a clean close event with the disconnect_probe_failed reason plus a logged warning — proven in BOTH directions (§11.4.201(1)): a raising probe closes the stream cleanly AND a normally-

connected client still streams uninterrupted. Prefer reusing the shared helper introduced by BOB-139 rather than a third copy of the logic (§11.4.251).

Honest boundary (§11.4.6): the production probe-failure rate is UNKNOWN — nobody has measured how often `is_disconnected()` actually raises. This is filed on the inconsistency and the missing diagnosis, not on a measured incident rate.

Affected scope / file-scope manifest: download-proxy/src/api/routes.py (~line 167, `stream_theme`)

Reproduction / context: Monkeypatch `Request.is_disconnected` to raise, open `/theme/stream`, observe the generator dies with an uncaught traceback rather than emitting event: close with reason `disconnect_probe_failed` as `streaming.py` now does.

Acceptance criteria: Same probe-failure discipline as `streaming.py` (clean close + `disconnect_probe_failed` reason + logged warning), verified both directions, reusing the BOB-139 helper rather than a third copy.